

Clinica Odontológica "Sonrisa Feliz"

Materia: Paradigma Orientado a Objetos

Entrega: Segunda entrega — Servicios, Repositorios, Consola CRUD

1. Arquitectura implementada

El sistema se organizó en cuatro capas con responsabilidades claramente separadas:

Capa	Paquete	Responsabilidad
Modelo	<code>Modelo</code>	Entidades del dominio: <code>Paciente</code> , <code>Odontologo</code> , <code>Turno</code> , <code>Domicilio</code>
Repositorio	<code>Repositorio</code>	Persistencia en memoria (HashMap), acceso a datos
Servicio	<code>Servicio</code>	Lógica de negocio y validaciones
Presentación	<code>Presentacion</code>	Interfaz por consola, lectura de inputs, menús

Esta separación es la base sobre la que se aplican los patrones GRASP y los principios SOLID descritos a continuación.

2. Patrones GRASP aplicados

2.1 Controller (Controlador)

Concepto: El controlador es el primer objeto que recibe y coordina las operaciones del sistema, orquestando la respuesta sin contener lógica de negocio propia.

Aplicación: Las clases `ServicioPaciente`, `ServicioOdontologo` y `ServicioTurno` cumplen el rol de Controller para cada entidad. Reciben las solicitudes de la capa de presentación, coordinan los repositorios y aplican las reglas de negocio, pero no interactúan directamente con la consola ni con la estructura interna de los datos.

Ejemplo concreto — `ServicioPaciente.registrarPaciente()`:

```
public void registrarPaciente(Paciente paciente) {
    Paciente existente = repoPaciente.buscarPorDni(paciente.getDni());
    if (existente != null) {
        throw new RuntimeException("DNI duplicado");
    }
    repoPaciente.guardar(paciente);
}
```

El servicio orquesta la consulta al repositorio y la validación de negocio. La presentación no sabe nada de esa lógica: solo llama al servicio dentro de un `try/catch` y muestra el mensaje de error al usuario.

Ejemplo concreto — `ServicioTurno.agendarTurno()`:

```
public void agendarTurno(Paciente p, Odontologo o, LocalDate fecha, LocalTime hora) {
    if (repoPaciente.buscarPorId(p.getId()) == null)
        throw new RuntimeException("Paciente no encontrado");
    if (repoOdontologo.buscarPorId(o.getId()) == null)
        throw new RuntimeException("Odontologo no encontrado");
    if (!fecha.isAfter(LocalDate.now()))
        throw new RuntimeException("La fecha del turno debe ser futura");
    if (repoTurno.existeTurno(o, fecha, hora))
        throw new RuntimeException("El odontologo ya tiene turno en ese horario");
    Turno turno = new Turno(p, o, fecha, hora, EstadoTurno.PENDIENTE);
    repoTurno.guardar(turno);
}
```

El servicio coordina cuatro validaciones de negocio antes de delegar la persistencia al repositorio. Ninguna de esas validaciones existe en la presentación.

2.2 Information Expert (Experto en Información)

Concepto: La responsabilidad de responder una pregunta o ejecutar una operación debe asignarse a la clase que tiene la información necesaria para hacerlo.

Aplicación: Las entidades del dominio implementan los métodos que solo ellas pueden responder, ya que son las únicas con acceso directo a sus propios datos.

Ejemplo concreto — `Turno.esFuturo()`:

```
public boolean esFuturo() {
    if (this.fecha == null || this.hora == null) return false;
    if (this.fecha.isAfter(LocalDate.now())) return true;
    return this.fecha.isEqual(LocalDate.now()) && this.hora.isAfter(LocalTime.now());
}
```

Solo `Turno` puede responder si es futuro: es quien tiene `fecha` y `hora`. Este método lo usan `ServicioPaciente` y `ServicioOdontologo` para validar si se puede eliminar una entidad con turnos activos, sin necesidad de acceder directamente a los atributos del turno.

Ejemplo concreto — `Turno.estaDisponible()`:

```
public boolean estaDisponible() {
    return this.estado == EstadoTurno.PENDIENTE;
}
```

La entidad encapsula la lógica de estado. Los servicios la consultan sin conocer cómo está implementada internamente.

Ejemplo en repositorio — `RepositorioPaciente.buscarPorDni()`:

```
public Paciente buscarPorDni(String dni) {
    for (Paciente p : pacientes.values()) {
        if (p.getDni().equals(dni)) return p;
    }
    return null;
}
```

El repositorio es el experto en el conjunto de pacientes: es el único que sabe cómo están almacenados y cómo buscar por atributos que no son la clave primaria del mapa.

Ejemplo adicional — `RepositorioPaciente.listarPorLocalidad()`:

```
public List<Paciente> listarPorLocalidad(String localidad) {
    List<Paciente> pacientesLocalidad = new ArrayList<>();
    for (Paciente p : pacientes.values()) {
        Domicilio dom = p.getDomicilio();
        if (dom != null && dom.getLocalidad() != null && dom.getLocalidad().equals(localidad)) {
            pacientesLocalidad.add(p);
        }
    }
    return pacientesLocalidad;
}
```

Ningún servicio ni menú necesita saber que los pacientes están en un `HashMap`: ni iterar sobre ellos directamente. El repositorio expone una consulta con semántica de negocio (`listarPorLocalidad`) y oculta completamente la estructura interna.

2.3 Creator (Creador)

Concepto: La responsabilidad de crear un objeto A debe asignarse a la clase B que contiene, agrega, registra o usa instancias de A de manera cercana.

Aplicación: Los servicios crean los objetos del dominio porque son quienes tienen el contexto de negocio para validar los datos antes de la creación, y quienes los entregan al repositorio para su almacenamiento.

Ejemplo concreto — creación de `Turno` en `ServicioTurno.agendarTurno()`:

```
Turno turno = new Turno(p, o, fecha, hora, EstadoTurno.PENDIENTE);
repoTurno.guardar(turno);
```

El servicio crea el `Turno` con estado inicial `PENDIENTE` porque es quien conoce esa regla de negocio: todo turno nuevo comienza pendiente. La presentación solo recolecta los datos del usuario y los pasa al servicio; no instancia entidades directamente.

3. Principios SOLID demostrados

3.1 S — Responsabilidad Única (Single Responsibility)

Cada clase tiene exactamente una razón para cambiar:

- `RepositorioPaciente`: cambia solo si cambia la forma de almacenar pacientes.
- `ServicioPaciente`: cambia solo si cambian las reglas de negocio sobre pacientes.
- `MenuPaciente`: cambia solo si cambia la interfaz de usuario para pacientes.

Por ejemplo, la regla "un paciente no puede eliminarse si tiene turnos futuros activos" vive únicamente en `ServicioPaciente.eliminarPaciente()`. Si esa regla cambia, solo se modifica ese método. Ni el repositorio ni la presentación saben que esa regla existe.

El sistema es extensible sin modificar el código existente gracias a dos mecanismos:

Interfaz `IRepository<T>`:

```
public interface IRepository<T> {
    void guardar(T obj);
    T buscarPorId(Long id);
    void actualizar(T obj);
    void eliminar(Long id);
    List<T> listar();
}
```

Si en el futuro se necesita persistencia en base de datos, se crea `RepositorioPacienteDB` implements `IRepository<Paciente>` sin tocar `ServicioPaciente`. El servicio depende de la abstracción, no de la implementación concreta.

Jerarquia de `Odontologo`: agregar una nueva especialidad como `Periodoncista` es crear una sola subclase nueva. No se modifica `Odontologo`, ni los servicios, ni los repositorios, ni los menús existentes.

3.3 L — Sustitución de Liskov (Liskov Substitution)

`OdontologoGeneral`, `Ortodoncista` y `Endodoncista` extienden `Odontologo`. En cualquier lugar del sistema que recibe un `Odontologo`, puede recibir cualquiera de sus subclases sin alterar el comportamiento esperado.

```
// MenuTurno - trabaja con Odontologo sin importar la subclase concreta
Odontologo odontologo = servicioOdontologo.buscarOdontologoId(odontologo);
System.out.println("Duración estimada: " + odontologo.calcularDuracionTurno() + " minutos.");
```

No se usa `instanceof` para bifurcar según el tipo concreto. El método correcto se resuelve en tiempo de ejecución según el objeto real.

3.4 D — Inversión de Dependencia (Dependency Inversion)

Los servicios reciben sus repositorios por constructor en lugar de crearlos internamente, lo que desacopla las capas:

```
public ServicioPaciente(RepositorioPaciente repoPaciente, RepositorioTurno repoTurno) {
    this.repoPaciente = repoPaciente;
    this.repoTurno = repoTurno;
}
```

El ensamblado de todas las dependencias ocurre en un único lugar: `Main.java`. Esto refleja el principio de que los módulos de alto nivel no deben construir sus propias dependencias.

4. Herencia y polimorfismo

4.1 Decisión de diseño

Se identificó que los odontólogos tienen comportamiento diferenciado según su especialidad, particularmente en la duración de cada turno. La alternativa descartada era modelar esto con un atributo `String especialidad` y un `switch` en el servicio:

```
// alternativa descartada - viola OCP e Information Expert
switch (odontologo.getEspecialidad()) {
    case "General" -> duracion = 30;
    case "Ortodoncista" -> duracion = 45;
    case "Endodoncista" -> duracion = 60;
}
```

Este enfoque viola OCP porque agregar una especialidad obliga a modificar el `switch`. También viola Information Expert porque la duración es una responsabilidad de la especialidad, no del servicio. Y no produce polimorfismo real: son condicionales disfrazados.

La solución adoptada es una jerarquía de clases con método abstracto:

```
Odontologo (abstract)
├── OdontologoGeneral    ~ calcularDuracionTurno() = 30 min
├── Ortodoncista         ~ calcularDuracionTurno() = 45 min
└── Endodoncista        ~ calcularDuracionTurno() = 60 min
```

4.2 Implementación

`Odontologo` declara el método abstracto que cada especialidad debe implementar obligatoriamente:

```
public abstract class Odontologo {
    // atributos y métodos comunes a todas las especialidades
    public abstract int calcularDuracionTurno();
}
```

Cada subclase lo implementa con su propia lógica:

```
public class Ortodoncista extends Odontologo {
    public Ortodoncista(String nombre, String apellido, String matricula) {
        super(nombre, apellido, matricula);
    }

    @Override
    public int calcularDuracionTurno() { return 45; }

    @Override
    public String toString() {
        return super.toString() + "\nEspecialidad: Ortodoncista (45 min)";
    }
}
```

4.3 Usos polimórficos en el sistema

Duración del turno en `MenuTurno.agendar()`:

```
System.out.println("Duración estimada: " + odontologo.calcularDuracionTurno() + " minutos.");
```

La presentación obtiene la duración correcta sin saber qué tipo concreto de odontólogo es. El mismo código funciona para las tres especialidades.

Filtro por especialidad en `RepositorioOdontologo.buscarPorEspecialidad()`:

```
public List<Odontologo> buscarPorEspecialidad(Class<?> especialidad) {
    List<Odontologo> resultado = new ArrayList<>();
    for (Odontologo o : odontologos.values()) {
        if (especialidad.isInstance(o)) resultado.add(o);
    }
    return resultado;
}
```

`isInstance()` es el equivalente dinámico de `instanceof`. Permite filtrar por especialidad sin hardcodear ningún tipo concreto en el método — si se agrega `Periodoncista`, el método sigue funcionando sin modificaciones.

Este patrón es el mismo que el profesor demostró con `estabilidad()` en el TP de Biblioteca: el método polimórfico elimina la necesidad de preguntar qué tipo de objeto es.

4.4 Herencia de comportamiento común

Las tres subclases heredan de `Odontologo` los atributos (`id`, `nombre`, `apellido`, `matricula`), los constructores vía `super()`, y los métodos comunes (`getNombreCompleto()`, `equals()`, `hashCode()`). Solo sobrescriben lo que es diferente: `calcularDuracionTurno()` y la línea de especialidad en `toString()`. Esto evita duplicación de código y garantiza consistencia entre especialidades.

5. Decisiones de diseño adicionales

Estado de turno como enum: `EstadoTurno` modela el ciclo de vida del turno (`PENDIENTE` - `CONFIRMADO` - `COMPLETADO` o `CANCELADO`) como un tipo cerrado y seguro. Los servicios transicionan el estado a través de métodos con nombre semántico (`confirmarTurno`, `cancelarTurno`, `completarTurno`), evitando que la presentación haga `setEstado()` directamente.

Método auxiliar `buscarOLanzar()` en `ServicioTurno`: Se extrajo el patrón repetido de buscar por id y lanzar excepción si es null a un método privado, aplicando el principio de no repetir lógica (DRY) dentro de la misma clase.

```
private Turno buscarOLanzar(Long id) {
    Turno turno = repoTurno.buscarPorId(id);
    if (turno == null) throw new RuntimeException("Turno no encontrado");
    return turno;
}
```

ID en entidades: Se optó por mantener el contador estático en las propias entidades para simplicidad de implementación en esta etapa. Una mejora futura sería delegar la asignación de IDs al repositorio, que sería el Creator más puro según GRASP.